

promise; take the response from that promise and format it as a message from the bottom. You can get the bot's actual text response from `res.result.fulfillment.speech`. As you did with the user message, make this an instance of the message class and then call the `update()` method on it.

```
return this.client.textRequest(msg)
    .then(res => {
      const speech = res.result.fulfillment.speech;
      const botMessage = new Message(speech, 'bot');
      this.update(botMessage);
    });
```

Putting the service to use (*chat-dialog.component.ts*)

Now switch back to the `chat-dialog.component.ts` file. You need to import the `ChatService`, `Message` class from `chat.service.ts` file, and `Observable` class and `Scan` operator from `RxJs`.

```
import { Observable } from 'rxjs/Observable';
import 'rxjs/add/operator/scan';
```

To arrange your messages in an observable array, you can tap into the behaviour subject that we created in the service. You need to set a variable for messages and `formValue`, which we'll see once we get to the HTML.

```
messages: Observable<Message[]>;
formValue: string;
```

You need to add `ChatService` in the constructor and convert that behaviour subject into an observable array, which you can do with the `Scan` operator by calling the behaviour subject as `Observable`.

```
constructor(public chat: ChatService) { }

ngOnInit() {
  // appends to array after each new message is added to
  feedSource
  this.messages = this.chat.conversation.asObservable()
```

Scanning gives you the current value as well as the accumulated total in that behaviour subject. You can concatenate two values into a new array together. The end result is an observable array that you can loop over in HTML with the `async pipe`.

```
.scan((acc, val) => acc.concat(val) )
```

By taking the form value that will set up with `ng-model` in HTML, we can call the `converse` method from the service. This is just an event handler that will bind to some events in HTML.

```
sendMessage() {
  this.chat.converse(this.formValue);
  this.formValue = '';
}
```

Letting a user send messages (*chat-dialog.component.html*)

Head over to the `chat-dialog.component.html` file. You need to loop over that `Observable` list of messages. You can do that with `ngFor` and the `async pipe`, which will give you access to the message object. You can use `ngClass` to distinguish the bot messages from the actual human messages. The 'from' messages will be sent by the bot and 'to' messages will be sent by the user.

```
<ng-container *ngFor="let message of messages | async">

  <div class="message" [ngClass]="{ 'from': message.sentBy
  === 'bot',
  'to': message.sentBy === 'user' }">
    {{ message.content }}
  </div>

</ng-container>
```

The other step is to set up a form so the users can actually type their messages. The quickest way to take care of that is with `ngModel`. You can set up a form input with `ngModel` set to the `formValue`. When the user clicks the `Enter` button, your code will fire the event handler that will send the message to `Dialogflow`. You can set up a button that fires the event handler on a click as well, in case the user doesn't have an `Enter` button.

```
<label for="nameField">Your Message</label>

<input [(ngModel)]="formValue" (keyup.enter)="sendMessage()"
type="text">

<button (click)="sendMessage()">Send</button>
```

Congratulations on building your first chatbot! `Dialogflow` gives your customers new points of interaction to connect with your brands by building appealing conversational interfaces powered by AI. You can connect with your customers on your Web page, application, Google Assistant, Microsoft Cortana, Amazon Alexa, Facebook Messenger, and other popular platforms and devices. **END** 🐧

By: Aditya Modi

The author is the CEO of TOPS Infosolutions, a leading software development company. He provides innovative solutions to organisations worldwide that address their business and engineering problems. LinkedIn: <https://in.linkedin.com/in/aditya-modi-2309053>